ASSIGNMENT 1

NAME: JANANIPRIYA G
REG.NO: 192324041
SUBMISSION DATE:9/11/26
SLOT:A
SUBJECT:JAVA PROGRAMMING
COURSE CODE:CSA0903

CO1: Java fundamentals and object-oriented programming – data types, operators, control

statements, arrays, classes and objects, constructors, static members, inheritance, method

overriding, polymorphism, abstract classes, interfaces, packages, access modifiers and

encapsulation

Assignment Title: Design, Implement and Validate an Object-Oriented Model for a Campus

Smart-Parking System

TASK 1 – UNDERSTAND THE PROBLEM AND IDENTIFY THE OBJECTS

1.1 Problem Analysis

The Campus Smart-Parking System is designed to manage a multi-level parking facility used by students, staff, and visitors. The system contains multiple floors, and each floor has a fixed number of parking slots. It supports different vehicle types, including two-wheelers, cars, and electric vehicles.

When a vehicle enters the campus, the system must identify the vehicle, allocate a suitable free parking slot, generate a unique parking ticket, and record the entry time. When the vehicle leaves, the system must release the parking slot and calculate the parking fee based on the vehicle type and parking duration.

Staff and students with valid monthly passes are allowed to park without paying the parking fee, while visitors must pay the applicable charges. Electric vehicles may use charging slots, and charging fees are calculated separately.

The main objective is to develop an object-oriented Java program that manages parking operations efficiently, prevents invalid operations, and can be extended to support new vehicle categories and pricing rules.

1.2 Inputs and Outputs

Inputs:

- Vehicle registration number
- Vehicle type
- User category (student, staff, or visitor)
- Monthly pass validity
- Parking floor and slot availability
- Vehicle entry time
- Vehicle exit time
- Charging units consumed by electric vehicles

Outputs:

- Allocated parking floor and slot number
- Unique parking ticket number
- Recorded vehicle entry time
- Parking duration
- Calculated parking fee
- Charging fee, if applicable
- Parking slot availability after vehicle exit
- Error message when a parking operation is invalid

1.3 Candidate Classes and Their Responsibilities

Candidate Class	Responsibilities
ParkingFacility	Represents the complete parking facility and manages floors, parking slots, and slot allocation.
ParkingSlot	Represents an individual parking slot and maintains its slot number, floor number, and occupancy status.

Vehicle	Represents a vehicle and stores its registration number, vehicle type, and parking-related information.
TwoWheeler	Represents a two-wheeler and provides its specific parking fee calculation.
Car	Represents a car and provides its specific parking fee calculation.
ElectricCar	Represents an electric vehicle and supports charging-related operations.
ParkingTicket	Stores the unique ticket number, vehicle details, allocated slot, entry time, and exit time.
ParkingUser	Represents a student, staff member, or visitor using the parking facility.
MonthlyPass	Represents a valid monthly parking pass and determines whether the user is eligible for free parking.
ParkingFeeCalculator	Calculates the parking fee according to vehicle type, parking duration, and applicable rules.

These classes are required because each represents a separate responsibility in the parking system. ParkingFacility manages the overall parking area, ParkingSlot manages individual slot availability, Vehicle represents different vehicle categories, and ParkingTicket records each parking transaction. ParkingUser and MonthlyPass support user-specific parking rules, while ParkingFeeCalculator separates fee calculation from parking management.

1.4 Nouns and Verbs Identified from the Problem

Nouns identified as candidate classes:

Parking facility, floor, parking slot, vehicle, two-wheeler, car, electric vehicle, parking ticket, user, monthly pass, parking fee.

Verbs identified as methods:

- Allocate a parking slot – allocateSlot()
- Release a parking slot – releaseSlot()
- Register a vehicle – registerVehicle()
- Generate a parking ticket – generateTicket()

- Record entry time – recordEntryTime()
- Record exit time – recordExitTime()
- Calculate parking fee – calculateFee()
- Check slot availability – isAvailable()
- Validate monthly pass – validateMonthlyPass()
- Calculate charging fee – calculateChargingFee()

These methods represent the main operations required to manage vehicle entry, parking, exit, and payment.

1.5 Business Rules

The following business rules must be satisfied by the system:

1. A parking slot can contain only one vehicle at a time.
2. A vehicle must be allocated a suitable free slot before entering the parking facility.
3. Every parking ticket must have a unique ticket number.
4. A ticket number must never be reused.
5. The vehicle registration number must be valid and should not be duplicated among currently parked vehicles.
6. The entry time must be recorded when the vehicle enters the parking facility.
7. The exit time must be recorded when the vehicle leaves.
8. The parking duration must not be negative.
9. The parking fee must depend on the vehicle type and parking duration.
10. Students and staff with valid monthly passes are eligible for free parking.
11. Visitors must pay the applicable parking fee.
12. Electric vehicles using charging facilities must pay the applicable charging fee separately.
13. A parking slot must be released after the vehicle exits.
14. A vehicle cannot exit using an invalid or inactive parking ticket.
15. The system must handle a full parking facility without allocating an occupied slot.

These rules ensure correct parking operations, accurate fee calculation, and proper management of parking slots and tickets.

1.6 Assumptions

The following assumptions are made for the design:

1. The parking facility contains multiple floors, and each floor has a fixed number of parking slots.
2. Each parking slot has a unique slot number within its floor.
3. The system supports two-wheelers, cars, and electric vehicles.
4. Every vehicle entering the facility receives a parking ticket.
5. The ticket number is generated automatically and uniquely.
6. Parking duration is calculated using the recorded entry and exit times.
7. Students and staff must have a valid monthly pass to receive free parking.
8. Visitors are charged according to the applicable parking fee rules.
9. Electric vehicle charging is optional and is charged separately.
10. The initial design uses separate vehicle classes so that new vehicle categories can be added in the future.

1.7 Alternative Concept and Complexity Analysis

In the object-oriented design, each vehicle is represented by a separate Vehicle object. The object stores its registration number, vehicle type, and other vehicle-related information. Different vehicle types can be represented using subclasses such as TwoWheeler, Car, and ElectricCar.

If vehicles were stored as plain Strings in parallel arrays, the system would use arrays such as:

- regNo[] – stores vehicle registration numbers
- type[] – stores vehicle types
- entryTime[] – stores vehicle entry times

The information of a vehicle would be stored at the same index in all arrays.

Searching for a vehicle:

In both designs, searching for a vehicle by registration number requires checking the stored registration numbers one by one when a simple array or sequential search is used.

For n vehicles:

- Best-case time complexity: O(1)

- Worst-case time complexity: O(n)
- Space complexity: O(n) for storing vehicle records

In the object-oriented design, the registration number and related information are stored together inside a Vehicle object. This improves organization and makes the program easier to maintain.

In the parallel-array design, all vehicle information is stored separately. The programmer must ensure that the same index is used in every array. If one array is updated incorrectly, the vehicle information may become inconsistent.

Adding a new vehicle attribute:

In the object-oriented design, a new attribute can be added to the Vehicle class. The existing vehicle objects can then store the additional information in an organized manner.

In the parallel-array design, a new array must be created for the additional attribute. The programmer must maintain the same index across all arrays, increasing the possibility of errors.

Comparison:

Feature	**Object-Oriented Design**	**Parallel Arrays**
Data organization	Vehicle information is stored in objects.	Vehicle information is stored in separate arrays.
Vehicle search	O(n) using sequential search.	O(n) using sequential search.
Adding an attribute	Add a field to the Vehicle class.	Create and maintain another array.
Data consistency	Related data is grouped together.	Same-index consistency must be maintained.
Maintenance	Easier to extend and maintain.	More difficult when attributes increase.
Space complexity	O(n) for storing n vehicle objects and their fields.	O(n) for each attribute array; total remains O(n) for a fixed number of attributes.

1.8 Conclusion

The object-oriented approach is selected for the Campus Smart-Parking System because it organizes vehicle information and parking operations into separate classes

with clear responsibilities. It supports encapsulation, inheritance, and code reusability. Compared with parallel arrays, the object-oriented design is easier to maintain and extend when new vehicle categories, attributes, and parking rules are introduced.

TASK 2 – CHOOSE THE DATA REPRESENTATION

2.1 Selection of Java Data Types

The following Java data types are used to store parking information:

Field	Data Type
Floor number	int
Slot number	int
Vehicle registration number	String
Vehicle type	String
Entry and exit time	LocalDateTime
Parking duration	long
Parking fee	double
Slot availability	boolean
Ticket number	int

These data types are selected based on the type of information stored and the operations performed.

2.2 Representation of Floors and Slots

A jagged array String[][] is used to represent the parking floors and slots because each floor can have a different number of slots.

Example:

```
String[][] slots = {
   new String[20],
   new String[50],
   new String[100]
};
```

Each row represents a floor, and each column represents a parking slot. A null value indicates an available slot.

2.3 Type Conversion and Casting

Parking duration is calculated in minutes and converted into hours for fee calculation.

Integer division may produce an incorrect result:

double hours = 130 / 60; // Result: 2.0

Correct calculation:

double hours = 130 / 60.0; // Result: 2.1667

The system uses floating-point division and Math.ceil() to charge for every started hour and avoid incorrect fee calculations.

2.4 Reading Input

The Scanner class is used to receive input from the user, such as vehicle number, vehicle type, floor number, and slot number. It is suitable for interactive parking operations.

2.5 Alternative Concept and Complexity Analysis

Parking slots can also be represented using a one-dimensional array or an ArrayList<String[]>.

- **One-dimensional array:** Suitable when every floor has an equal number of slots. Different floor capacities require additional index calculations.
- **ArrayList:** Allows floors to be added dynamically and supports different slot capacities.
- **Jagged array:** Directly supports different numbers of slots on each floor.

Representation	Slot Access	Space Complexity
Jagged array	O(1)	O(S)
One-dimensional array	O(1)	O(S)
ArrayList of arrays	O(1)	O(S)

Here, S represents the total number of parking slots. Searching for an available slot may take O(S) time in the worst case.

2.6 Conclusion

The jagged array is selected because it supports different numbers of parking slots on each floor. Suitable Java data types and Scanner are used for storing and receiving

information. Floating-point division prevents errors in parking fee calculations. This representation makes the system simple, efficient, and flexible.

TASK 3 – Design the Control Logic

```
public class ParkingFacility {

  private final String[][] slots;

  public ParkingFacility(int[] slotsPerFloor) {

    slots = new String[slotsPerFloor.length][];

    for (int f = 0; f < slotsPerFloor.length; f++) {
      slots[f] = new String[slotsPerFloor[f]];
    }
  }

  public int[] allocate(String vehicleNo) {

    for (int f = 0; f < slots.length; f++) {

      for (int s = 0; s < slots[f].length; s++) {

        if (slots[f][s] == null) {

          slots[f][s] = vehicleNo;
```

```
            return new int[] {f, s};
          }
        }
    }

    return null;
}

public boolean release(int floor, int slot) {

    if (floor < 0 || floor >= slots.length) {
        return false;
    }

    if (slot < 0 || slot >= slots[floor].length) {
        return false;
    }

    if (slots[floor][slot] == null) {
        return false;
    }

    slots[floor][slot] = null;

    return true;
}

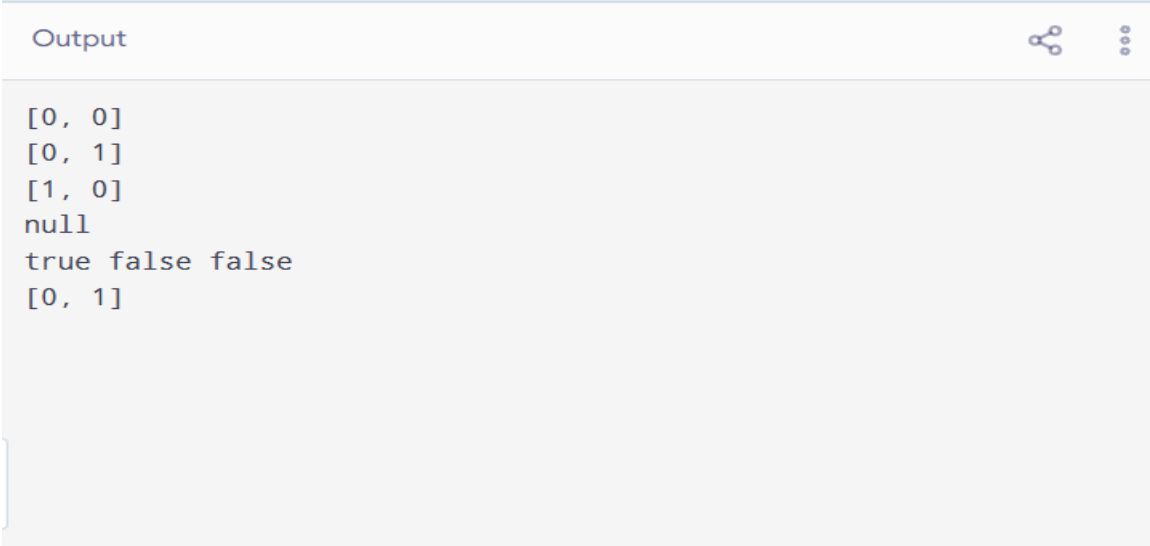
public static void main(String[] args) {
```

```
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});

        System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));

        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
                + p.release(5, 0));

        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```



TASK 4 – DESIGN CLASSES, OBJECTS AND CONSTRUCTORS

4.1 Class Design

The Campus Smart-Parking System is designed using different Java classes, each having a specific responsibility.

- **Vehicle:** Stores vehicle registration number and vehicle type.
- **ParkingSlot:** Stores floor number, slot number, and availability status.

- **ParkingFacility:** Manages parking floors and slot allocation.
- **ParkingTicket:** Stores ticket number, vehicle details, and entry and exit times.
- **ParkingUser:** Stores user details and pass information.

4.2 Objects

Objects are created from classes to represent real-world entities in the parking system.

For example:

- A Car object represents a car entering the parking facility.
- A ParkingSlot object represents an individual parking space.
- A ParkingTicket object represents the ticket issued to a vehicle.

4.3 Constructors

Constructors are special methods used to initialize objects when they are created. They have the same name as the class and do not have a return type.

Example:

```
class Vehicle {
  String regNo;
  String type;

  Vehicle(String regNo, String type) {
    this.regNo = regNo;
    this.type = type;
  }
}

public class Main {
  public static void main(String[] args) {
    Vehicle v1 = new Vehicle("TN01A1234", "Car");

    System.out.println(v1.regNo);
```

```
        System.out.println(v1.type);
    }
}
```

Output:

TN01A1234

Car

4.4 Types of Constructors

1. **Default Constructor:** Initializes an object with default values.
2. **Parameterized Constructor:** Initializes an object with values provided during object creation.

Constructors help initialize vehicle details, parking slot information, and ticket details correctly.

4.5 Alternative Concept and Complexity Analysis

Instead of using constructors, object values can be assigned separately after creating an object.

- Using constructors initializes all required values during object creation.
- Without constructors, values must be assigned separately using statements.
- Both approaches take O(1) time for initializing a fixed number of fields.
- Both use O(1) space per object for a fixed number of fields.

Constructors provide better organization and reduce the possibility of creating objects with missing information.

4.6 Conclusion

Classes and objects represent the real-world components of the Campus Smart-Parking System. Constructors initialize object data and make the program organized, reusable, and easy to maintain.

TASK 5 – APPLY STATIC MEMBERS AND THE OBJECT LIFE CYCLE

5.1 Static Variables

Static variables are used to store data shared by all objects of a class.

- ticketCounter: Generates unique ticket numbers for every parking ticket.
- totalOccupancy: Stores the total number of occupied parking slots.

These variables belong to the class rather than individual objects. Using instance variables would create separate counters for each object.

5.2 Static Methods and Static Block

A static method can be used to generate ticket numbers or display total parking occupancy. A static block is suitable for initializing common values when the class is loaded.

Example:

```
class ParkingTicket {
   static int ticketCounter = 1000;
   int ticketNo;

   ParkingTicket() {
      ticketNo = ++ticketCounter;
   }
}
```

5.3 Object Life Cycle and Garbage Collection

When a vehicle enters, a ParkingTicket object is created and stores its ticket number and entry details. When the vehicle leaves, the ticket is used to calculate the fee and release the parking slot.

After the ticket is no longer referenced by the program, it becomes eligible for garbage collection. Java automatically removes unused objects from memory.

5.4 Passing and Returning Objects

Objects are passed to methods using reference variables. Methods can also return objects.

Example:

```
ParkingTicket createTicket() {
   ParkingTicket ticket = new ParkingTicket();
   return ticket;
}
```

5.5 Alternative Concept and Complexity Analysis

If the ticket counter is an instance variable of ParkingFacility, each facility will have a separate counter. For example, the north and south gates may generate duplicate ticket numbers.

Using a static counter provides one shared sequence across all facility objects. If separate numbering is required for each gate, an instance counter can be used.

Counter Type	Space for k Facilities
Static counter	O(1)
Instance counter	O(k)

Generating each ticket takes O(1) time. Creating n tickets also takes O(n) total time. The static counter uses constant space, while instance counters require one counter per facility.

5.6 Conclusion

Static members are useful for maintaining common information such as ticket numbers and total occupancy. Garbage collection automatically manages unused ticket objects. Using static and instance members appropriately helps maintain an organized and efficient parking system.

TASK 6 – DESIGN THE INHERITANCE HIERARCHY

6.1 Superclass and Subclasses

Inheritance is used to represent different vehicle types in the parking system. Vehicle is the superclass, and TwoWheeler and Car are its subclasses. ElectricCar inherits from Car.

- **Vehicle:** Stores common details such as registration number and provides common methods.
- **TwoWheeler:** Inherits vehicle details and defines the two-wheeler parking rate.
- **Car:** Inherits vehicle details and defines the car parking rate.
- **ElectricCar:** Inherits car details and supports EV charging.

This design uses hierarchical and multilevel inheritance.

6.2 Inherited and Added Features

All subclasses inherit the registration number and common fee calculation methods from Vehicle. Each subclass provides its own parking rate. ElectricCar additionally implements charging-related behaviour.

6.3 Use of super Keyword

The super keyword is used to call the parent class constructor and initialize inherited fields.

Example:

```
class Vehicle {
  String regNo;

  Vehicle(String regNo) {
    this.regNo = regNo;
  }
}

class Car extends Vehicle {
  Car(String regNo) {
    super(regNo);
  }
}
```

6.4 Constructor Execution Order

When an ElectricCar object is created, constructors execute from parent to child:

1. Vehicle constructor executes.
2. Car constructor executes.
3. ElectricCar constructor executes.

This ensures that inherited data is initialized before the subclass-specific features.

6.5 Inheritance Hierarchy Diagram

```
      Vehicle
        |
    ------------
    |          |
```

TwoWheeler Car

|

ElectricCar

6.6 Alternative Concept and Complexity Analysis

Instead of inheritance, composition can be used by creating a VehicleType class that stores the vehicle type and parking rate. The Vehicle class can contain a VehicleType object.

Inheritance	**Composition**
Uses parent and child classes	Uses objects inside another class
New types require subclasses	New types can be added using VehicleType objects
Fee calculation takes O(1) time	Fee calculation takes O(1) time
Stores common data in the parent class	Stores a reference to VehicleType in each vehicle

Both approaches require O(1) time for fee calculation. Inheritance is suitable for the current vehicle hierarchy, while composition provides more flexibility when adding new vehicle types and changing rates.

6.7 Conclusion

Inheritance helps reuse common vehicle properties and methods. The super keyword initializes parent class data, while subclasses provide specialized behaviour. This hierarchy makes the parking system organized and easier to extend.

TASK 7 – OVERRIDE AND SPECIALISE BEHAVIOUR

7.1 Method Overriding

Method overriding allows subclasses to provide their own implementation of a parent class method. In the parking system, calculateFee() can be overridden to calculate fees based on vehicle type.

The @Override annotation is used to ensure the method correctly overrides the parent method.

```
class Vehicle {
   double calculateFee() {
```

```
        return 20;
    }
}

class Car extends Vehicle {
    @Override
    double calculateFee() {
        return 50;
    }
}
```

7.2 Use of final Keyword

The final keyword is used to prevent changes to important values, methods or classes. For example, a fixed registration number can be declared as final.

7.3 Object Class Methods

- toString() displays vehicle details.
- equals() compares vehicles using registration numbers.
- hashCode() generates a hash value based on the registration number.

7.4 Alternative Concept and Complexity Analysis

Instead of overriding, a single calculateFee() method can use a switch statement to identify the vehicle type and calculate the fee. Both approaches take O(1) time for a fixed number of vehicle types. Method overriding is easier to maintain because new vehicle types can have their own fee calculation.

7.5 Conclusion

Method overriding provides different fee calculations for different vehicle types and improves code reusability and maintenance.

TASK 8 – APPLY POLYMORPHISM

8.1 Compile-Time Polymorphism

Method overloading allows multiple methods with the same name but different parameters.

```
double calculateFee(int minutes) {
   return minutes * 0.5;
}

double calculateFee(double hours) {
   return hours * 30;
}
```

The compiler selects the correct method based on the arguments.

8.2 Runtime Polymorphism

A parent class reference can store objects of different subclasses. When calculateFee() is called, Java executes the overridden method of the actual object.

```
Vehicle[] vehicles = {
   new TwoWheeler("TN01A1"),
   new Car("TN02B2")
};

for (Vehicle v : vehicles) {
   System.out.println(v.calculateFee());
}
```

8.3 Dynamic Method Dispatch

If the reference type is Vehicle and the actual object is Car, Java executes the Car version of calculateFee() at runtime.

8.4 Alternative Concept and Complexity Analysis

Instead of polymorphism, instanceof checks and casting can identify each vehicle type. With k vehicle types, this may require up to O(k) checks. Dynamic method dispatch selects the correct overridden method in O(1) time and reduces lengthy conditional statements.

8.5 Conclusion

Polymorphism supports method overloading and overriding. It allows different vehicle objects to be handled through a common parent reference, making the parking system flexible and easy to maintain.